

Solução Ótima para 15-Puzzle

Exploração de Algoritmos A*, IDA* e Pattern Databases (PDB)

Projeto de Algoritmos em Grafos

Gabriel Davi & Giordani Andre

O Desafio do 15-Puzzle

1. Espaço de Estados Massivo

O 15-puzzle possui aproximadamente **10,4 trilhões de estados** alcançáveis. Para comparação, o 8-Puzzle possui apenas 181.440.

Neste nível de complexidade, buscas cegas como BFS ou DFS estouram o limite de tempo ou memória em segundos.

2. A Necessidade de Poda

Para encontrar a rota perfeitamente ótima num universo de trilhões de caminhos, é obrigatório o uso de algoritmos informados (Heurísticas) que "cortem" ramos inúteis sem sacrificar a precisão matemática.

Teoria das Inversões: Filtro de Viabilidade

Metade do Universo é Insolúvel

Se testássemos tabuleiros impossíveis, o algoritmo seria forçado a mapear exhaustivamente todo o espaço de busca na tentativa de encontrar um alvo inalcançável, causando o travamento do sistema.

Como o algoritmo se defende:

Lendo o tabuleiro da esquerda para a direita, uma "inversão" ocorre sempre que um número maior precede um menor. Cruzando a paridade dessas inversões com a posição da linha do espaço vazio, sabemos imediatamente se a resolução é possível.

Tem Solução: $(\text{Soma das Inversões} + \text{Linha do Vazio}) \equiv 0 \pmod{2}$

| Algoritmo A* Clássico

Como Funciona

O A* expande os caminhos escolhendo sempre o nó com o menor custo total previsto: $f(n) = g(n) + h(n)$.

Ele mantém uma "Fronteira" e uma tabela de nós já visitados, impedindo que o computador processe o mesmo tabuleiro duas vezes (Transposições).

O Inimigo: A Memória RAM

A sua complexidade espacial é **exponencial** $O(b^d)$. Ao tentar guardar milhões de tabuleiros na RAM para evitar as transposições, o A* causa um erro fatal de alocação de memória (`std::bad_alloc`) em instâncias difíceis do 15-Puzzle.

IDA* e a Economia de Memória

Iterative Deepening A*

Em vez de armazenar todos os nós explorados como o A*, o IDA* utiliza a técnica de **Busca em Profundidade (DFS)** guiada por limites rigorosos.

Complexidade Espacial $O(L)$:

O algoritmo carrega na RAM **apenas a linha reta do caminho atual**. Se o limite de movimentos é 50, existem no máximo 50 tabuleiros alocados na memória naquele momento.

O Preço do Recálculo:

Ao bater no limite e podar um caminho, o IDA* apaga o tabuleiro da memória. O trade-off é que, ao recomençar com um limite maior, ele gasta ciclos de CPU recalculando os mesmos nós iniciais.

| Abordagem 1: Distância de Manhattan



Cálculo Absoluto

Mede a distância ideal (horizontal e vertical) que cada peça precisa percorrer até a sua posição final.



Processamento Dinâmico

Calculado "on the fly" pela CPU a cada movimento, não ocupando nenhuma memória fixa na RAM.



Miopia Heurística

É excessivamente otimista. Ignora o fato de que num tabuleiro apertado, as peças precisarão desviar umas das outras.

| Aprimoramento: Conflito Linear

Engarrafamento de Peças

Se duas peças estão na mesma linha ou coluna, e as suas posições corretas exigem que elas se cruzem, ocorre um Conflito Linear.

+ **A Solução:** Somamos **+2 movimentos** à Distância de Manhattan para cada conflito detectado, forçando o algoritmo a orçar as manobras de desvio.

Admissibilidade Mantida: O cálculo continua perfeitamente seguro e incapaz de sobrestimar o custo real, mantendo a garantia de encontrar o caminho ótimo.

| Abordagem 2: Pattern Databases (PDB)

O "Oráculo" de Memória

Em vez de adivinhar o custo via CPU na hora do jogo, nós dividimos o tabuleiro em Grupos de Peças estáticos e mapeamos todo o universo através de uma Análise Retrógrada (de trás para a frente).

Consulta Instantânea

O algoritmo descobre exatamente quantos movimentos faltam para resolver um grupo específico com uma complexidade de tempo perfeita de **$O(1)$** .

O Ponto de Equilíbrio: A Matemática dos 5.5 MB

5.5 MB

Ocupação Exata na RAM por Grupo (6 Peças)

Permutação Combinatória (15-Puzzle): $16 \times 15 \times 14 \times 13 \times 12 \times 11 = 5.765.760$ estados possíveis.

Ao armazenar o custo de cada estado em um inteiro de 1 Byte (`uint8_t`), o consumo final na memória é rigorosamente de 5,63 MB. Dividimos o PDB em dois grupos para extrair o máximo de poda sem explodir a memória do computador.

Nota Comparativa (8-Puzzle): Embora o 8-Puzzle rastreie mais peças (8), o tabuleiro é menor (9 posições). A matemática da permutação ($9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2$) resulta em apenas **362.880 estados**, ocupando meros **~354 KB** de RAM na memória principal.

A Estratégia de Cálculo do PDB Híbrido

Formação do Custo PDB

Somamos o custo absoluto na RAM do **Grupo 1 (6 peças)** com o custo do **Grupo 2 (6 peças)**. Para as 3 peças restantes ("O Ponto Cego"), calculamos o Manhattan Puro em tempo real.

Otimização Extrema (O Juiz)

Para garantir que o IDA* faça a maior poda possível, o código avalia as duas abordagens ao mesmo tempo e escolhe sempre o palpite mais rigoroso utilizando `std::max(Total PDB, Total Clássico)`.

| A Regra de Ouro: Evitando o Imposto Duplo

Por que NÃO usamos Conflito Linear nas peças do PDB?

O Risco de *Double Counting* (Quebra de Admissibilidade).

Quando o PDB foi pré-calculado, os engarrafamentos físicos entre as peças daquele grupo já foram vivenciados e seus custos já estão embutidos na tabela.

Se a nossa função de Conflito Linear somasse +2 movimentos sobre peças do PDB, nós estaríamos multando a mesma manobra duas vezes. Isso geraria uma sobrestimativa, o que corrompe a matemática da Busca Heurística e faz o algoritmo descartar a solução ótima.

Resultados (8-Puzzle)

© Média de Movimentos Ótimos: ~22 movimentos

Métrica	Manhattan + Conflito	Pattern Database + Manhattan	Melhoria %
Nós Expandidos	~1.123	22	-98,04%
Tempo de Resolução	~0,12 ms	0,002 ms	-98,33%

Ambos garantiram o caminho ótimo perfeito. No 8-Puzzle, o tamanho diminuto do tabuleiro permite que a tabela PDB conheça o ambiente em sua totalidade, transformando a busca num resgate imediato na memória.

| Ganhos Temporais (15-Puzzle)

Tempo Total de Execução (100 Instâncias no IDA*)

Manhattan + Conflito

3.808 Segundos

Pattern Database +
Manhattan

2.914 Segundos

Apesar de pagar o custo fixo do *Trade-off* Inicial de I/O para carregar a tabela na RAM, a precisão absoluta do PDB poupou **~15 minutos inteiros** (23% de redução) da CPU executando cálculos inúteis e retrocessos.

O Impacto da Poda (15-Puzzle)

🗺️ Caminho Ótimo Encontrado: Média de 53,05 Movimentos por Instância

Volume de Nós Expandidos (Média de 100 Instâncias)

Manhattan + Conflito

22,2 Milhões de Nós

Pattern Database +
Manhattan

12,8 Milhões de Nós

Redução massiva de **42%** no esforço de processamento mantendo a solução perfeita.

O Triunfo Final do A*

Tempo Total PDB A*: 2.576 Segundos (~11,6% mais rápido que IDA* PDB)

O algoritmo A* registrou o tempo mais veloz de todo o projeto. Por quê?

A Poda Salvou a RAM:

Com a Heurística Clássica, os 22 milhões de nós explodiam a memória no A*. Ao reduzirmos a árvore quase pela metade com o PDB, a Fila de Prioridade coube confortavelmente na memória principal.

Eficiência Máxima:

Operando dentro da RAM, o A* usou o seu arsenal de rastreio de transposições para cortar caminhos repetidos instantaneamente e nunca precisou recalcular iterações passadas (o maior defeito do IDA*).

Obrigado!